CSCI 6461 | TOPIC 1

Introduction to Computing Systems & Binary Foundations

Abstraction, stored-program organization, data representation, and processor performance

Dr. J. Burns
Computer Systems Architecture — AArch64 Reference Platform

PART 1

Architecture Foundations & Abstraction

What Is Computer Architecture?

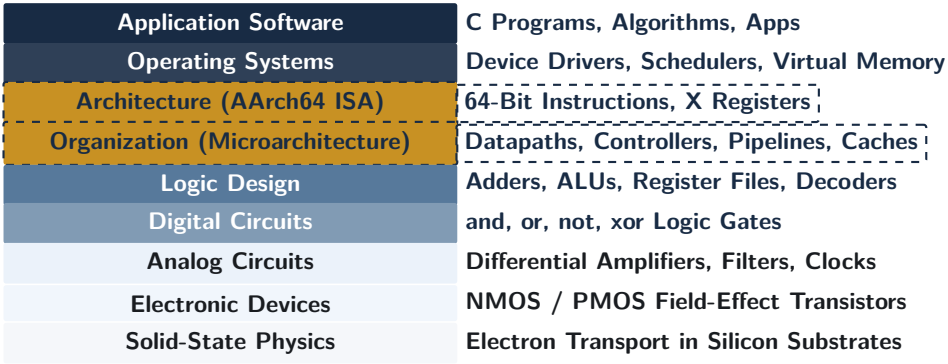
- Computer architecture connects **software-visible behavior** with **hardware implementation**.
- Hardware provides an interface that software can rely on.
- The implementation of that interface determines performance and efficiency.
- Architectural design involves correctness, performance, power, area, and cost.

Course Approach

Design approach: RISC principles.
Reference ISA: AArch64.

The 9 Levels of Computer Abstraction

Computers manage immense complexity by enforcing clean hierarchical interfaces:



When Abstractions Leak

Software View

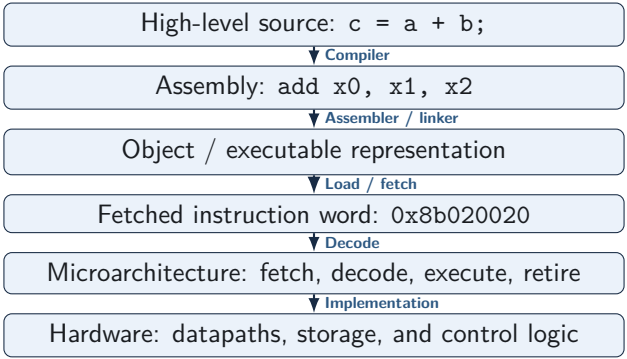
- Variables, arrays, branches
- Function calls, data types

Architectural Effect

- Register allocation, cache misses
- Branch prediction, stack frames
- Overflow and sign extension

Performance-sensitive software often exposes lower-level machine behavior.

From Source Code to Execution



The ISA is the boundary between program representation and processor execution.

ISA and Microarchitecture

Instruction Set Architecture

- Instruction encodings & behavior
- Registers `x0`–`x30`, `sp`, PSTATE
- Memory and exception behavior

Microarchitecture

- Pipeline organization, superscalar width
- In-order or out-of-order execution
- Branch prediction, cache structures

Different processors can implement the same ISA using different microarchitectures.

RISC Design Principles and AArch64 Encoding

- RISC designs emphasize regular instruction formats and explicit load/store architecture.
- AArch64 instructions are fixed-width **32-bit words**, aligned on 4-byte boundaries.
- Destination (`Rd`) and source operands (`Rn`, `Rm`) use fixed 5-bit register-number fields.

Example: 64-Bit Addition (`add x0, x1, x2` $\Longrightarrow$ `0x8b020020`)

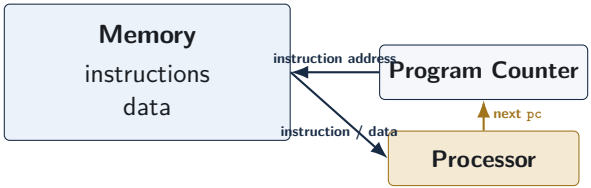
Encoding maps `Rd = x0`, `Rn = x1`, and `Rm = x2` within the uniform 32-bit word.

PART 2

Stored-Program Organization & Memory

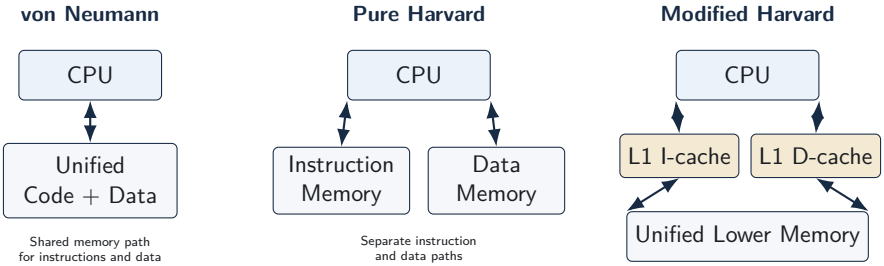
The Stored-Program Concept

- A stored-program computer keeps **instructions and data in memory as binary information**.
- The processor does not inherently "know" that a memory word is code or data; interpretation depends on how that address is used.
- The **Program Counter (pc)** identifies the address of the next instruction to be fetched.



The stored-program idea makes software possible: changing memory contents can change both the data being processed and the instructions being executed.

Memory Topologies: von Neumann, Harvard, Modified Harvard



- Modern high-performance processors commonly use a **modified Harvard organization**: separate first-level instruction/data caches with a unified software-visible address space.
- The key architectural question is not just "where is memory?" but **which paths instruction fetch and data access use**.

Modified Harvard and Newly Generated Code

- A JIT compiler or runtime system may **generate machine instructions by writing bytes as data**.
- Those writes travel through the data side of the memory hierarchy.
- The instruction side may still hold an older copy of the same address, so simply jumping to the new bytes may be unsafe.

Conceptual Synchronization

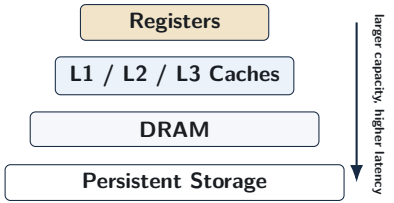
Make data writes visible → synchronize → discard stale instruction copies → execute

The exact AArch64 cache-maintenance and barrier instructions are system-software details that will be revisited after caches and the memory hierarchy are developed.

Memory Hierarchy: Why Multiple Levels Exist

Design Tension

- Small storage can be very fast.
- Large storage is cheaper per bit but slower.
- No single technology gives maximum speed, capacity, and low cost simultaneously.



Why the Hierarchy Works: Locality

Temporal locality: recently used data is likely to be used again.
Spatial locality: nearby data is likely to be used soon.

The Memory Wall

- Processor execution capability has improved much faster than off-chip DRAM latency.
- A cache hierarchy attempts to keep the processor working on nearby copies of frequently used instructions and data.
- When locality fails and a request must go farther down the hierarchy, execution may stall for many cycles.

Illustrative Latency Scale

- Registers: roughly a few pipeline resources away
- L1 cache: typically only a few cycles
- Lower-level caches: tens of cycles
- DRAM: often hundreds of processor cycles

Exact latencies are implementation-dependent; the architectural lesson is the large gap between nearby and distant storage.

PART 3

Binary Representation & Arithmetic

Bits and Interpretive Context

- Hardware stores binary bit patterns; **meaning comes from context**.
- The stored-program model therefore permits the same memory bytes to be interpreted differently depending on how they are used.

Interpretation Perspective	**Meaning of 32-bit Word** `0x8b020020`
AArch64 Machine Instruction	`add x0, x1, x2`
Unsigned 32-bit Integer	$2{,}332{,}164{,}128_{10}$
Signed 32-bit Integer	$-1{,}962{,}803{,}168_{10}$

Bits do not carry a built-in “type”; instructions and software conventions determine the interpretation.

Positional Binary and Hexadecimal

- In positional binary, each bit contributes a power of two (Value $= \sum_{i=0}^{n-1} b_i 2^i$).
- Hexadecimal is a compact notation for binary because one hex digit represents exactly four bits.

Example

$$10110110_2 = 128 + 32 + 16 + 4 + 2 = 182_{10} = \texttt{0xb6}$$

Endianness: Mapping Multi-Byte Values to Memory

- Endianness determines which byte of a multi-byte value occupies the lowest memory address.
- In little-endian storage, the least-significant byte is placed first.

Store 32-bit value `0x12345678` starting at address `0x1000`

Little-Endian Rule

Lowest address → least-significant byte

Why Two's Complement?

- A signed representation should support positive and negative values while keeping arithmetic hardware simple.
- Two's complement provides:
  - one representation of zero,
  - the same binary adder for signed and unsigned addition,
  - simple subtraction through addition of a negated operand.

Signed Interpretation

For an n-bit two's-complement number, Value $= -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i$.

Give the most-significant bit a negative weight, and add the remaining positive bit weights normally.

Constructing a Negative Two's-Complement Number

Example: Represent -5 in 8 Bits

$+5 = 00000101$

Invert all bits: 11111010

Add 1: 11111011

Therefore, $-5 = 11111011$

- The bit pattern can also be evaluated directly $(-128 + 64 + 32 + 16 + 8 + 2 + 1 = -5)$.
- An n-bit two's-complement integer ranges from -2^{n-1} to $2^{n-1} - 1$.

Unified Adder Datapath and Subtraction

- The same n-bit adder can operate on bit patterns interpreted as either signed or unsigned integers.
- Two's-complement negation gives $-B = \sim B + 1$, so subtraction becomes $A - B = A + (\sim B) + 1$.

Sign Extension and Zero Extension

- When a smaller value is widened, the upper bits must be filled in a way that preserves the intended numeric meaning.

Signed Widening

Sign extension replicates the source sign bit.
Example: `ldrsb` preserves a negative 8-bit value when widened.

Unsigned Widening

Zero extension fills upper bits with zeros.
Example: `ldrb` preserves the unsigned magnitude of an 8-bit value.

Why It Matters

Using the wrong extension changes the interpreted value and can invalidate comparisons, bounds checks, or address calculations.

AArch64 NZCV Condition Flags

Arithmetic instructions can update status flags in PSTATE:

Flag	Name	Evaluation Meaning
N	Negative	Most-significant result bit is 1
Z	Zero	All result bits are zero
C	Carry	Unsigned carry-out (add) or no-borrow (sub)
V	Overflow	Signed mathematical result is outside the destination range

Same Bits, Different Questions

For $1111_2 + 0001_2$ in 4 bits, the stored result is 0000_2.
Unsigned interpretation: $15 + 1 = 16$ produces carry-out ($C = 1$).
Signed interpretation: $-1 + 1 = 0$ does not overflow ($V = 0$).

Signed vs. Unsigned Conditional Branching

After `cmp x0, x1`, the processor has conceptually subtracted the operands and set NZCV.

Branch	Context	Flag Condition	Meaning
`b.eq` / `b.ne`	Universal	$Z = 1$ / $Z = 0$	Equal / Not Equal
`b.lt` / `b.ge`	Signed	$N \neq V$ / $N = V$	Less Than / Greater or Equal
`b.lo` / `b.hs`	Unsigned	$C = 0$ / $C = 1$	Lower / Higher or Equal

Why the Mnemonic Depends on Interpretation

The same 64-bit pattern can represent a negative signed integer or a very large unsigned integer. The correct branch condition follows the program's intended interpretation, not the raw bits alone.

PART 4

Quantitative Processor Performance

Defining Performance: Latency and Throughput

- For a fixed workload, Performance $= \frac{1}{\text{Execution Time}}$.
- **Latency:** time required to complete one task.
- **Throughput:** number of tasks completed per unit time.

IPC is one measure of instruction throughput inside a processor, not a universal measure of application-task throughput.

Dynamic Instruction Count and Average CPI

- **Dynamic Instruction Count (IC):** the actual number of instructions executed for a particular run.
- **Cycles Per Instruction (CPI):** workload-average cycles per retired instruction ($\text{CPI} = \frac{\text{Total Clock Cycles}}{\text{Dynamic Instruction Count}}$).
- Different instructions, cache misses, dependencies, and branch mispredictions can contribute different delays.

CPI Is Not a Fixed Property of the Processor

A processor can exhibit different CPI values on different programs, inputs, and memory-access patterns.

The Iron Law of Processor Performance

The Fundamental CPU Performance Equation

$$T_{\text{CPU}} = \text{Instruction Count (IC)} \times \text{CPI} \times T_{\text{clk}} = \frac{\text{IC} \times \text{CPI}}{f_{\text{clk}}}$$

Component	Influenced By	Examples
IC	Algorithm, compiler, ISA	instruction selection, code structure
CPI	Pipeline, dependencies, memory hierarchy	stalls, speculation, cache misses
T_{clk}	Circuit paths, pipeline organization	critical-path delay, implementation choices

Important

These factors are **not independent**. A design change that improves one factor can worsen another.

Iron Law: Comparative Evaluation Example

Machine A

- IC $= 1.0 \times 10^9$
- CPI $= 2.0$
- $f_{\text{clk}} = 4.0$ GHz

$$T_A = \frac{1.0 \times 10^9 \times 2.0}{4.0 \times 10^9} = \mathbf{0.50}\ \text{s}$$

Machine B

- IC $= 1.2 \times 10^9$
- CPI $= 0.8$
- $f_{\text{clk}} = 2.5$ GHz

$$T_B = \frac{1.2 \times 10^9 \times 0.8}{2.5 \times 10^9} = \mathbf{0.384}\ \text{s}$$

$$\text{Speedup} = \frac{T_A}{T_B} = \frac{0.50}{0.384} \approx \mathbf{1.30}\times$$

The equation compares measured assumptions; the numbers alone do not prove which microarchitectural feature caused the difference.

The Megahertz Myth and Speedup

- A higher clock frequency does not guarantee better performance because IC and CPI also matter.
- **Speedup** compares old and new execution times (Speedup $= \frac{T_{\text{old}}}{T_{\text{new}}} = \frac{\text{Performance}_{\text{new}}}{\text{Performance}_{\text{old}}}$).
- If runtime drops from 10 s to 4 s, Speedup $= \frac{10}{4} = 2.5\times$.

Amdahl's Law: Why Partial Optimization Is Limited

- Suppose only fraction f of execution time can be accelerated.
- Even a huge improvement to that fraction cannot eliminate the time spent in the unaffected portion $(1 - f)$.

Amdahl's Law

$$S_{\text{overall}} = \frac{1}{(1 - f) + \frac{f}{S_{\text{enhanced}}}}$$

Worked Example

For $f = 0.60$ and $S_{\text{enhanced}} = 10$:

$$S_{\text{overall}} = \frac{1}{0.40 + 0.06} \approx 2.17\times$$

Limiting Case

If that 60% became infinitely fast:

$$S_{\max} = \frac{1}{0.40} = 2.5\times$$

Benchmarking and Architectural Trade-Offs

- Simple metrics such as MIPS are unreliable across different ISAs because instruction counts and instruction meanings differ.
- Benchmark suites such as SPEC CPU use representative workloads to compare complete systems more meaningfully.

Trade-Offs Preview

Later topics will quantify how choices such as pipeline depth, issue width, and cache capacity can improve one performance factor while increasing complexity, latency, power, or another Iron-Law term.

Review and Self-Test Questions

Topic 1 Summary: Architecture & Memory

- **Abstraction Layers:** Systems span from high-level software down to physical devices, with the ISA forming the key software/hardware contract.
- **Stored-Program Model:** Instructions and data are represented as bits in memory; `pc` directs instruction fetch.
- **Modified Harvard Organization:** Separate first-level instruction/data paths can coexist with a unified software-visible address space.
- **Memory Hierarchy:** Multiple storage levels exploit temporal and spatial locality to reduce average access time.

Topic 1 Summary: Representation & Performance

- **Two's Complement:** Signed integers use a representation that supports one zero and a unified addition/subtraction datapath.
- **Condition Flags:** NZCV distinguishes unsigned carry behavior from signed overflow behavior.
- **The Iron Law:** CPU execution time depends jointly on instruction count, average CPI, and clock period.
- **Amdahl's Law:** Overall speedup is limited by the fraction of execution time that is not improved.

Self-Test Diagnostic Questions: Architecture & Binary

1. **ISA vs. Microarchitecture:** Categorize the following:
   - Number of visible registers (`x0`–`x30`).
   - Size of the L1 instruction cache.
   - Binary machine encoding of the `add` instruction.
2. **Two's Complement Construction:** Show the steps required to encode -37 as an 8-bit two's-complement value.
3. **Carry vs. Overflow:** In a 4-bit ALU evaluating $1001_2 + 1100_2$, determine the output sum, Carry flag (C), and Signed Overflow flag (V).

Self-Test Questions: Performance & Memory

1. **Iron Law Analysis:** A compiler optimization decreases dynamic instruction count by 20%, but increases average CPI by 15%. Does performance improve? Calculate the exact speedup.
2. **Amdahl's Law Application:** Memory stalls account for 70% of execution time. In order to achieve an overall speedup of $2.5\times$, what must the speedup of the memory portion be?
3. **Generated Code and Split Caches:** Why can a runtime system that writes new machine instructions need synchronization before those instructions are safely fetched and executed?

Looking Ahead

Next Lecture Preview

Our Next Topic: RISC vs. CISC: Instruction Set Design Principles & Trade-offs

- **Complexity Distribution:** Examining how different Instruction Set Architectures distribute complexity between software and hardware.
- **Key Trade-offs:** Evaluating instruction encoding, decode complexity, pipeline efficiency, and quantitative performance impact.